Overfitting and Cross Validation

高怡宣
陽明大學 生物醫學影像暨放射科學系

1. Overfitting
2. Cross Validation

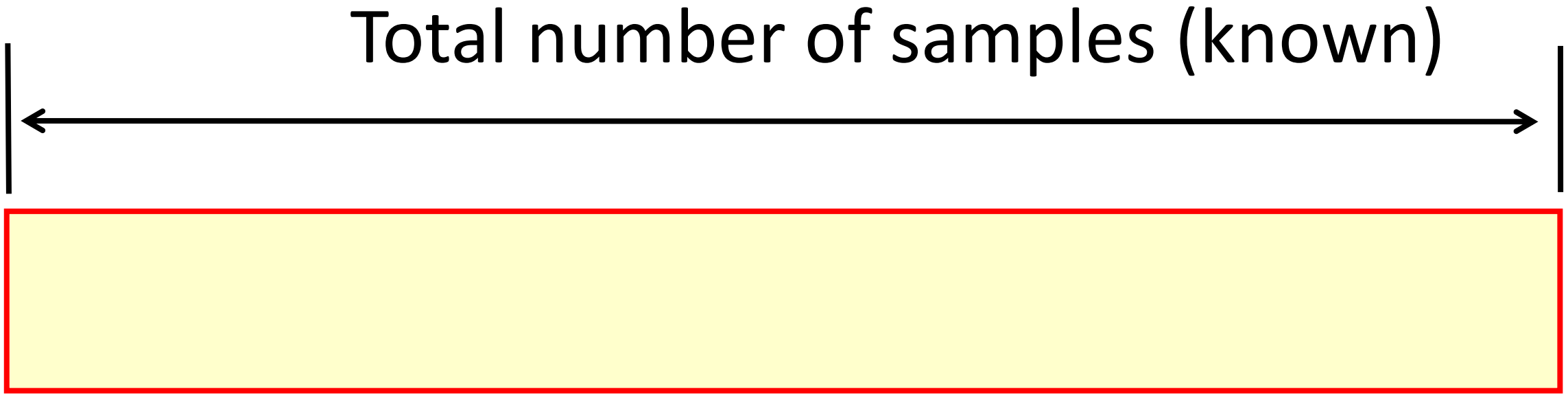
Overfitting:

過適現象(過度擬合)的觀念對機器學習很重要的。在學習過程太久或範例太少時，預測訓練範例結果的表現增加時，應用在未知資料的表現則變更差。

在統計和機器學習中，為了避免過適現象，須要使用額外的技巧（如提早停止、交叉驗證），以指出何時會有更多訓練，而沒有導致更好的一般化預測。

https://zh.wikipedia.org/wiki/%E9%81%8E%E9%81%A9

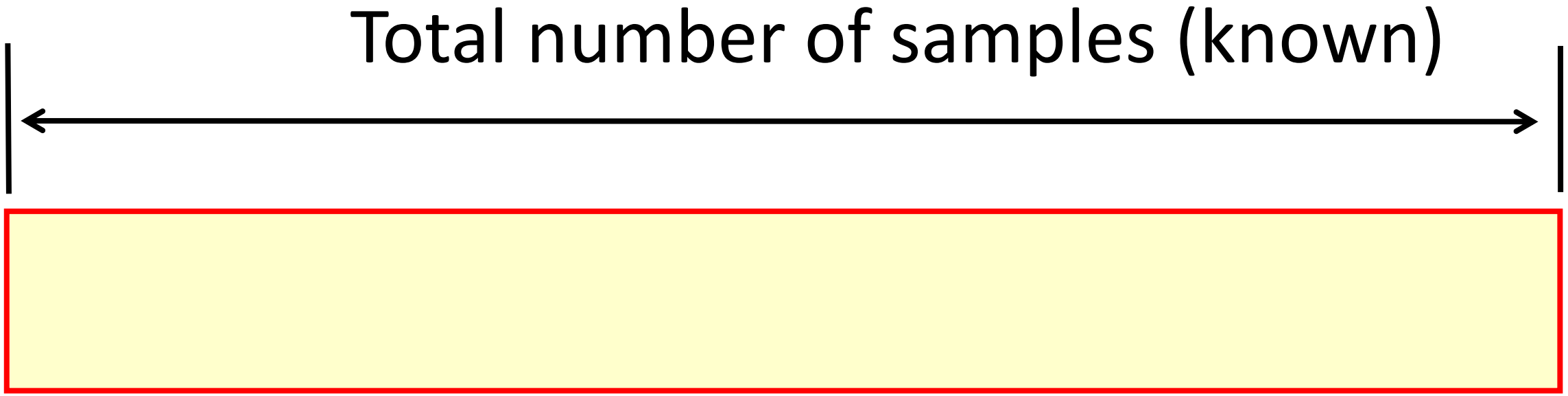
Overfitting



Unknown data

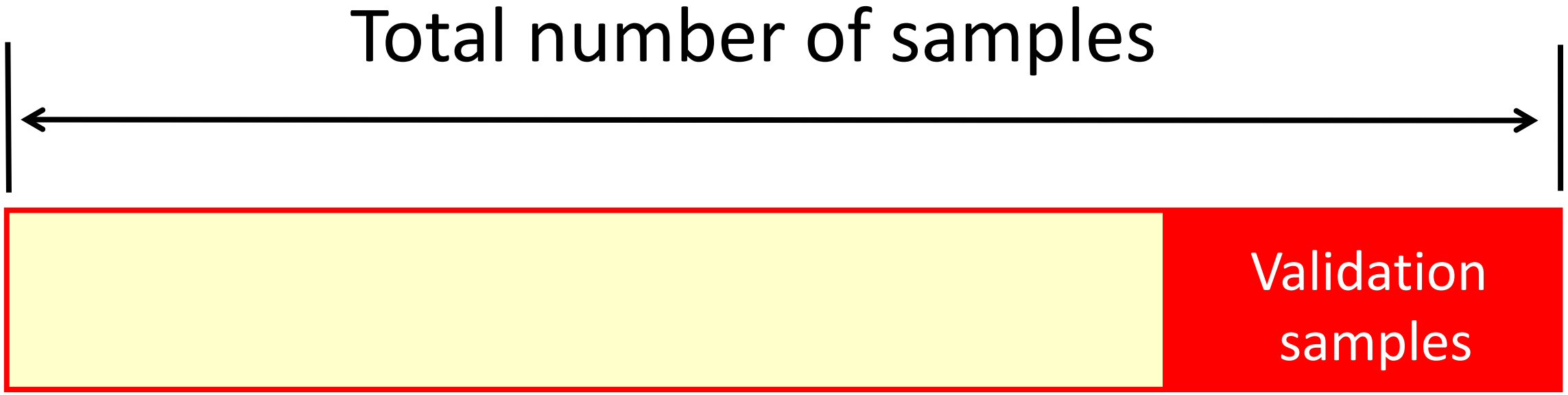
Reference: LECTURE 13: Cross-validation, Introduction to Pattern Analysis, Ricardo Gutierrez-Osuna, Texas A&M University

Overfitting



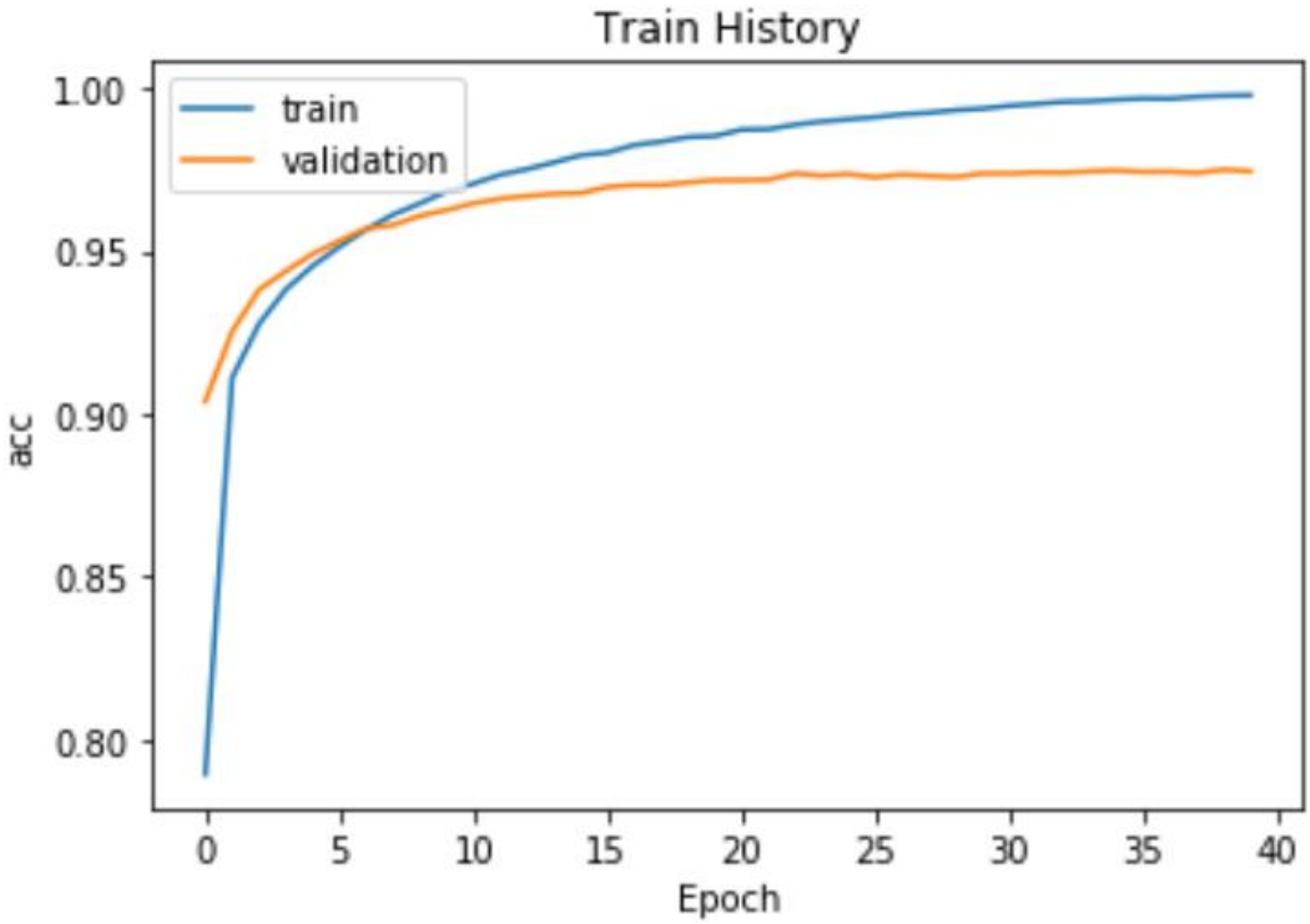
Reference: LECTURE 13: Cross-validation, Introduction to Pattern Analysis, Ricardo Gutierrez-Osuna, Texas A&M University

Overfitting

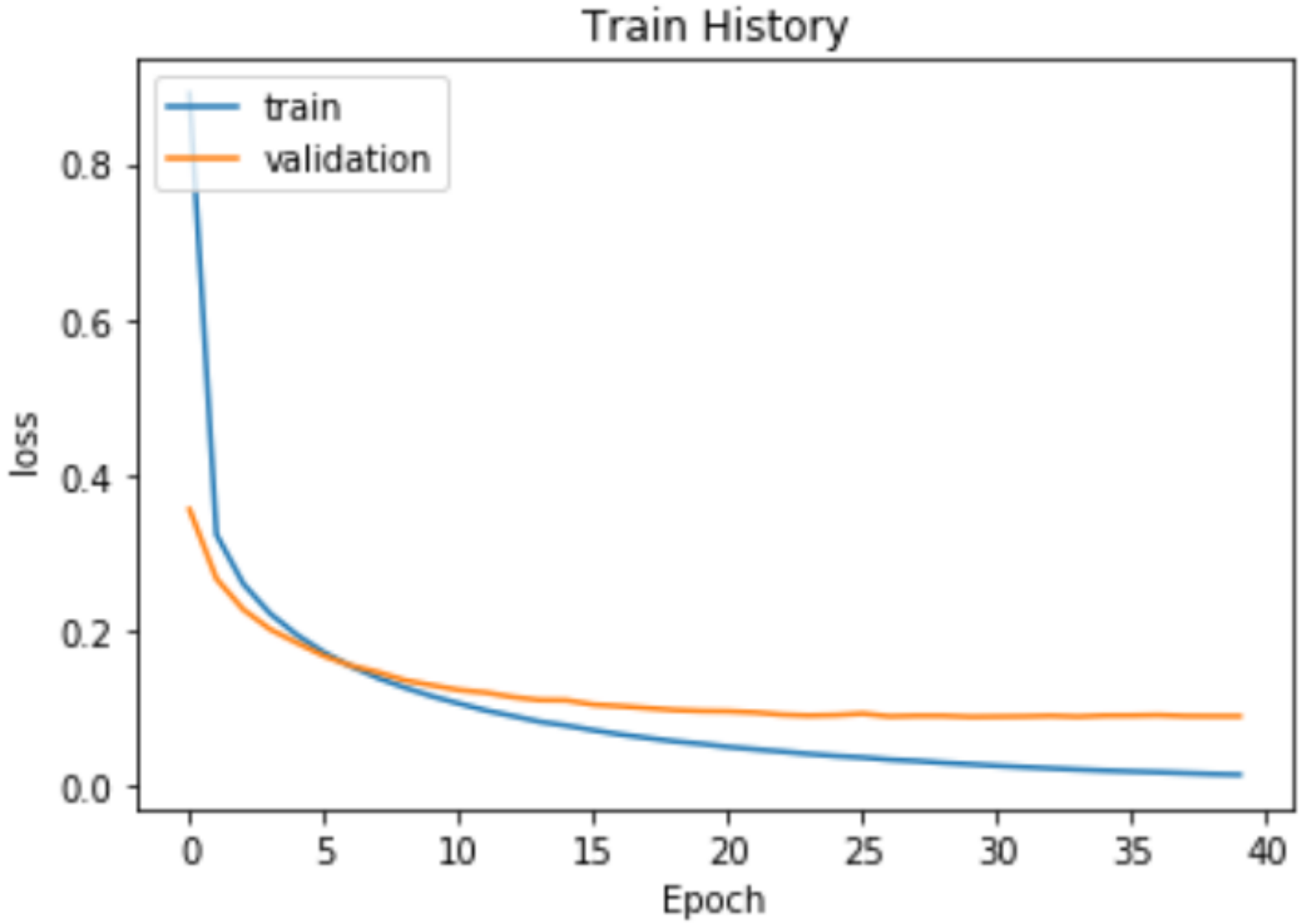


Reference: LECTURE 13: Cross-validation, Introduction to Pattern Analysis, Ricardo Gutierrez-Osuna, Texas A&M University

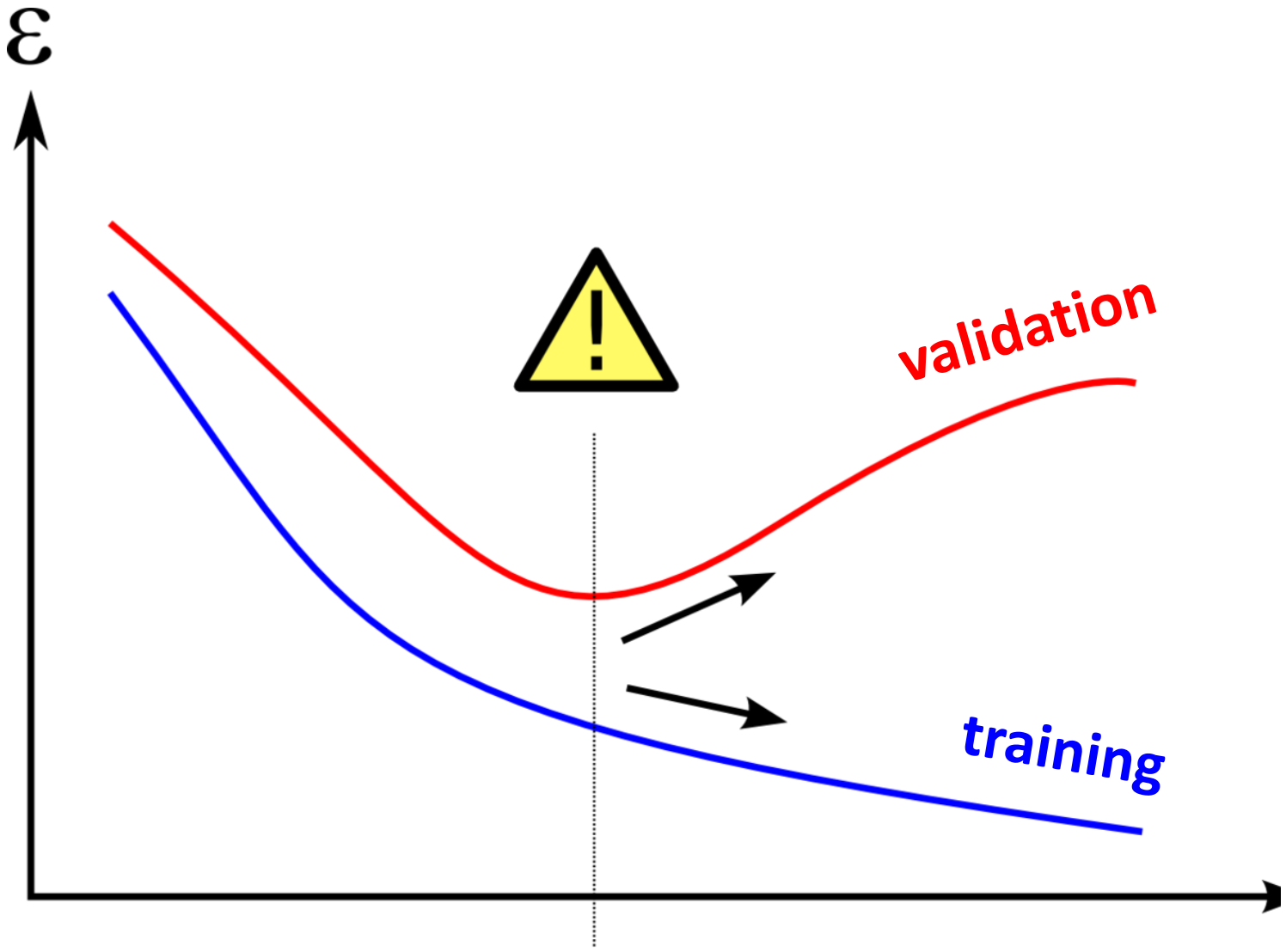
Overfitting



Overfitting

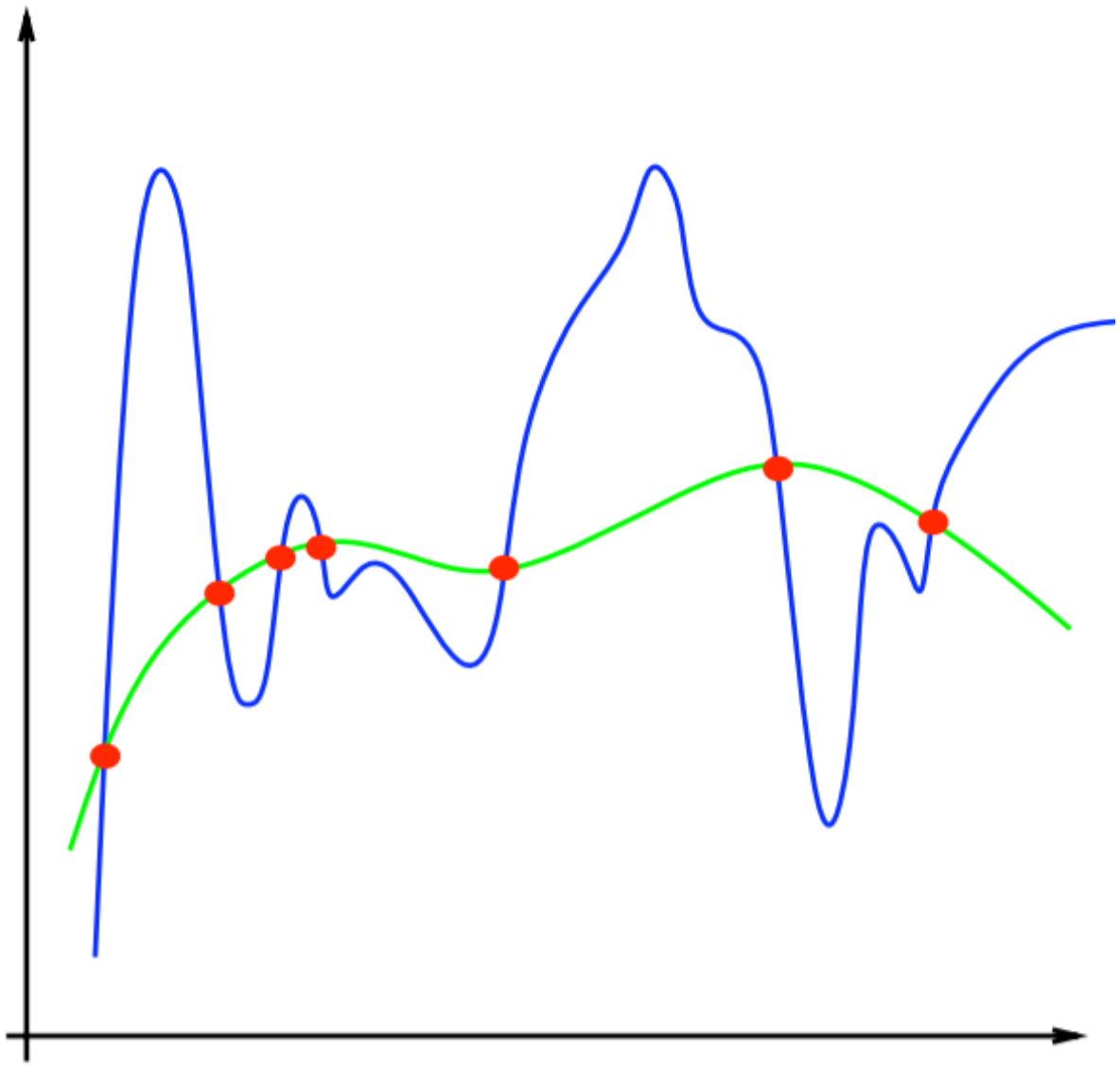


Overfitting



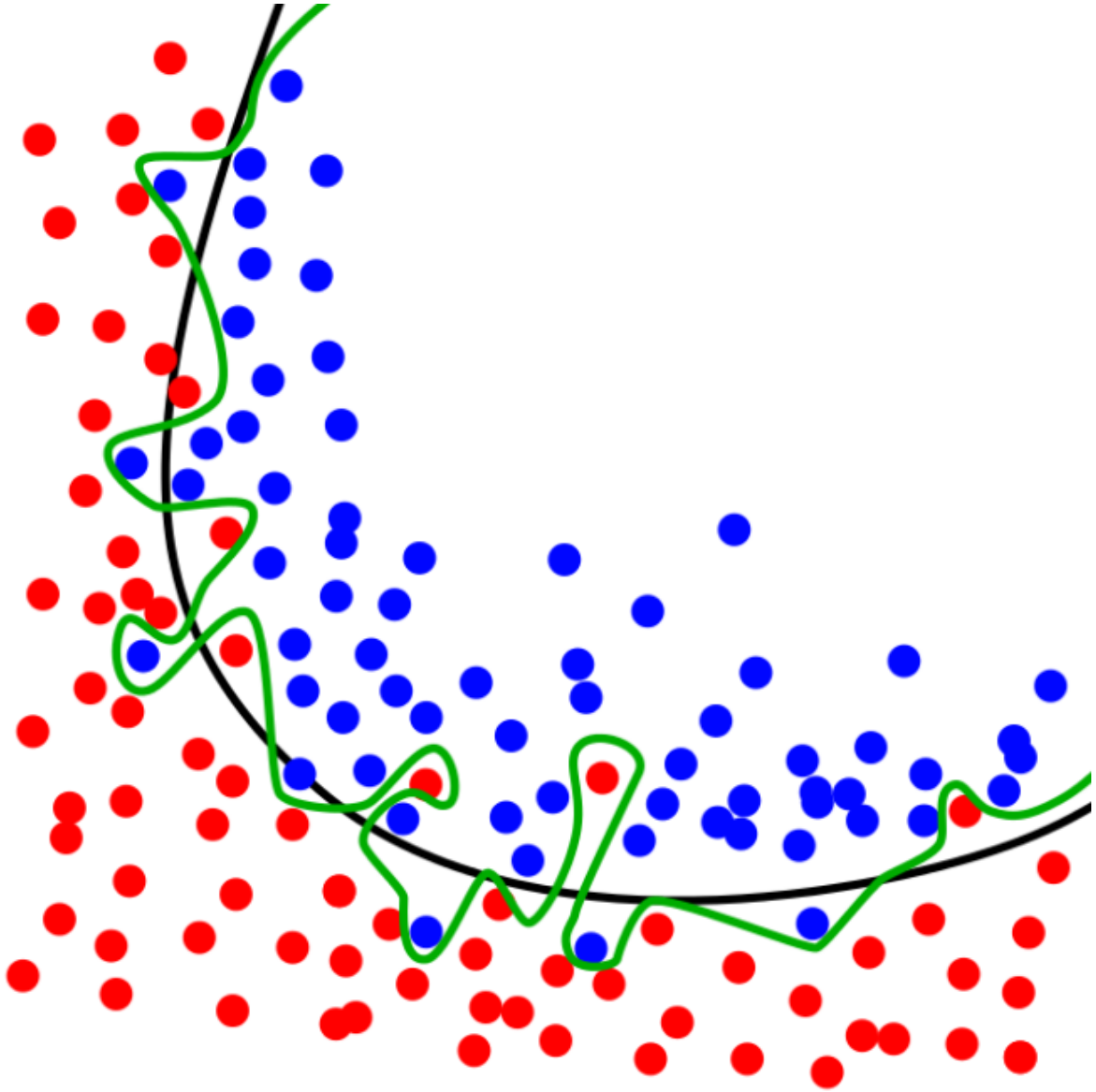
CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=263773

Overfitting



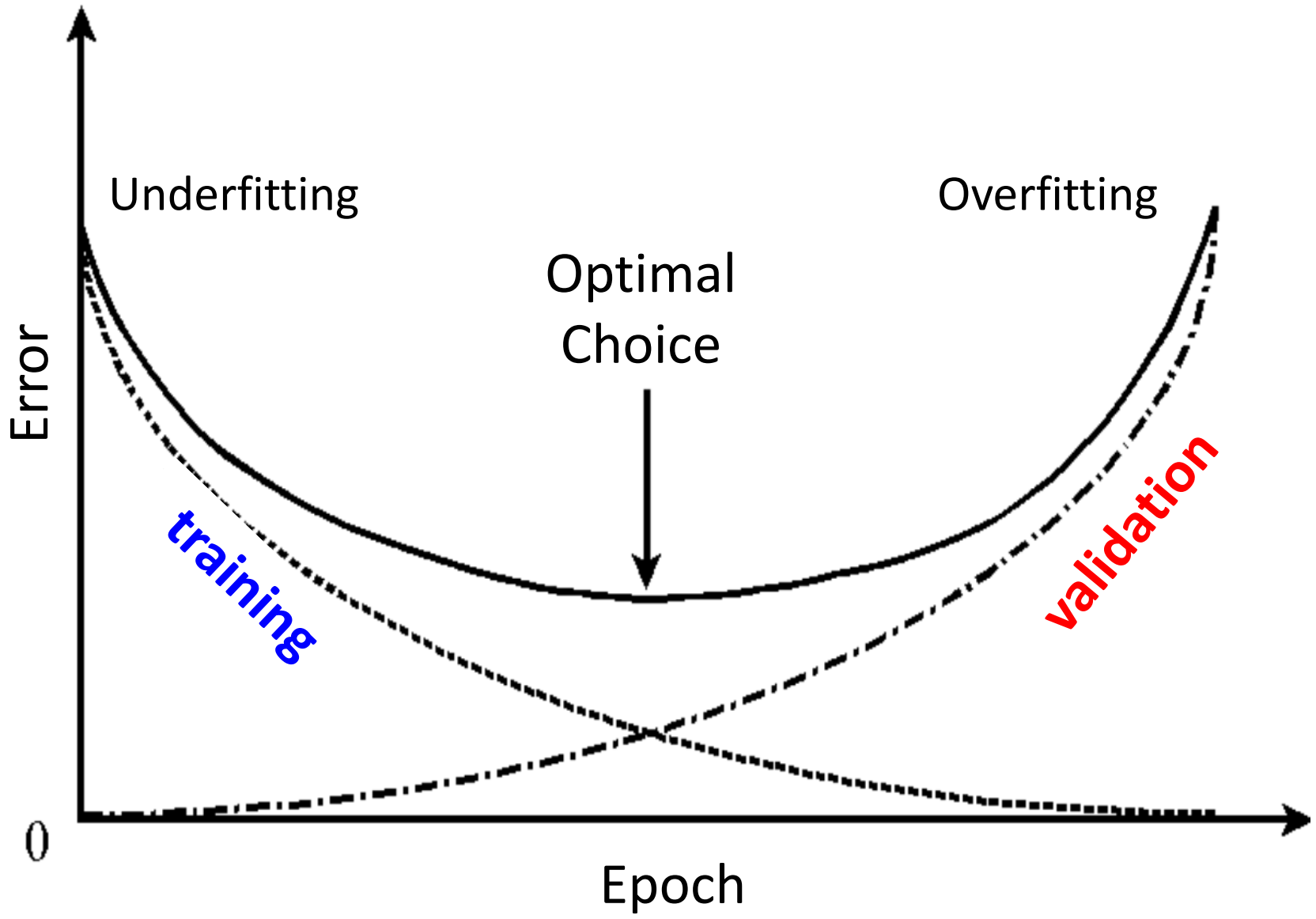
By Tomaso Poggio - http://www.mit.edu/~9.520/spring12/slides/class02/class02.pdf

Overfitting



By Chabacano - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=3610704

Overfitting



https://commons.wikimedia.org/wiki/File:MultivariateVoorspelling.png

1. Overfitting
2. Cross Validation

Cross Validation:

交叉驗證，是一種統計學上將數據樣本切割成較小子集的實用方法。一開始的子集被稱為訓練集，而其它的子集則被稱為驗證集或測試集。交叉驗證是在訓練階段定義一組用於「測試」模型的數據集，減少過度擬合的問題。

它對於防範根據數據建議的測試假設是非常重要的， 特別是當後續的樣本是危險、成本過高或科學上不適合搜集。

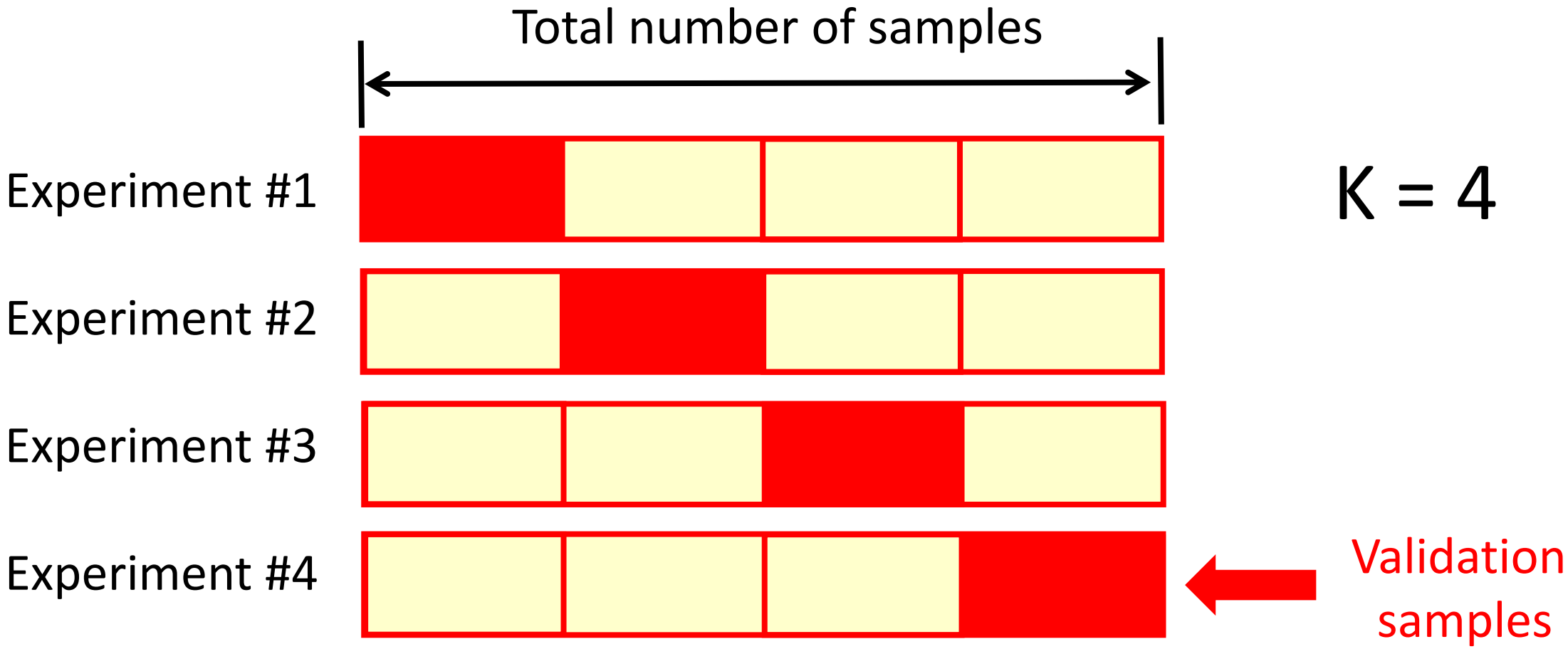
https://zh.wikipedia.org/wiki/%E4%BA%A4%E5%8F%89%E9%A9%97%E8%AD%89

Cross Validation:

1. K-Fold Cross-validation
2. Random Subsampling
3. Leave-one-out Cross Validation

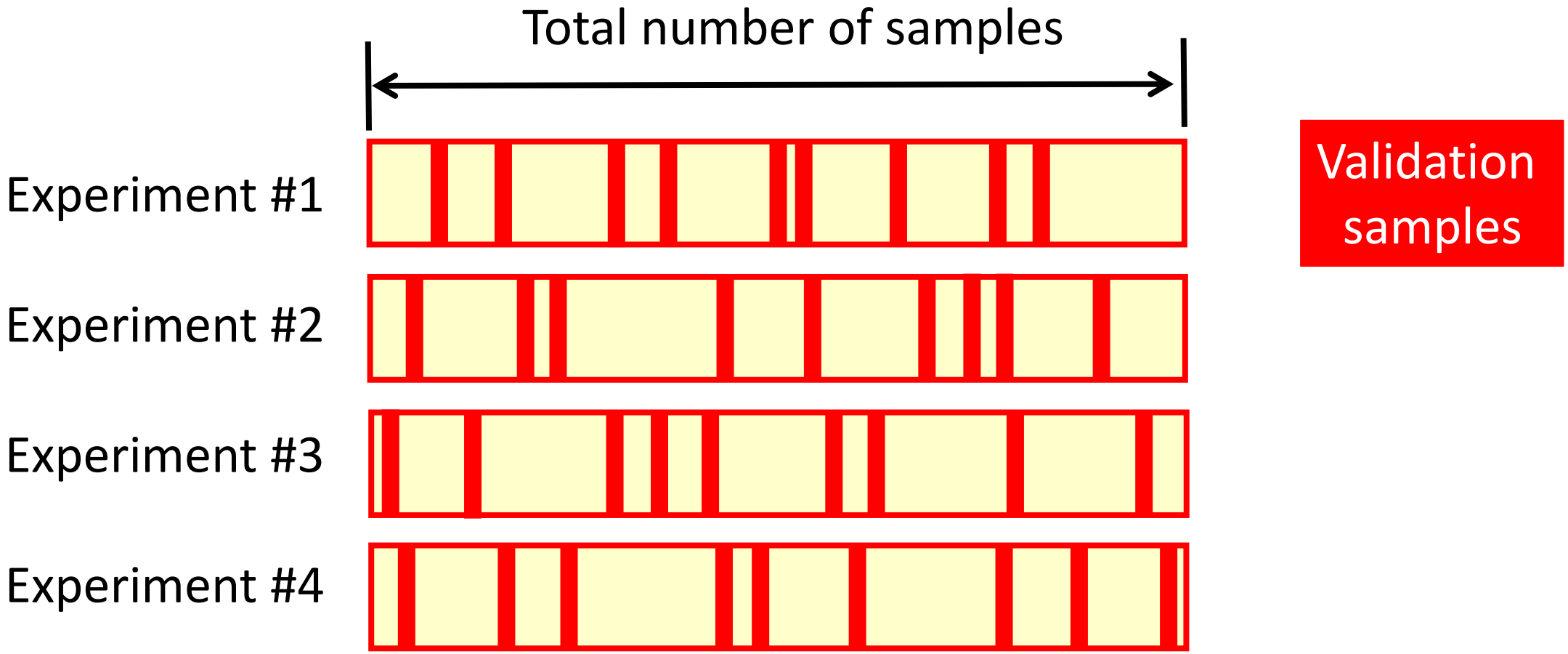
Reference: LECTURE 13: Cross-validation, Introduction to Pattern Analysis, Ricardo Gutierrez-Osuna, Texas A&M University

K-fold cross validation



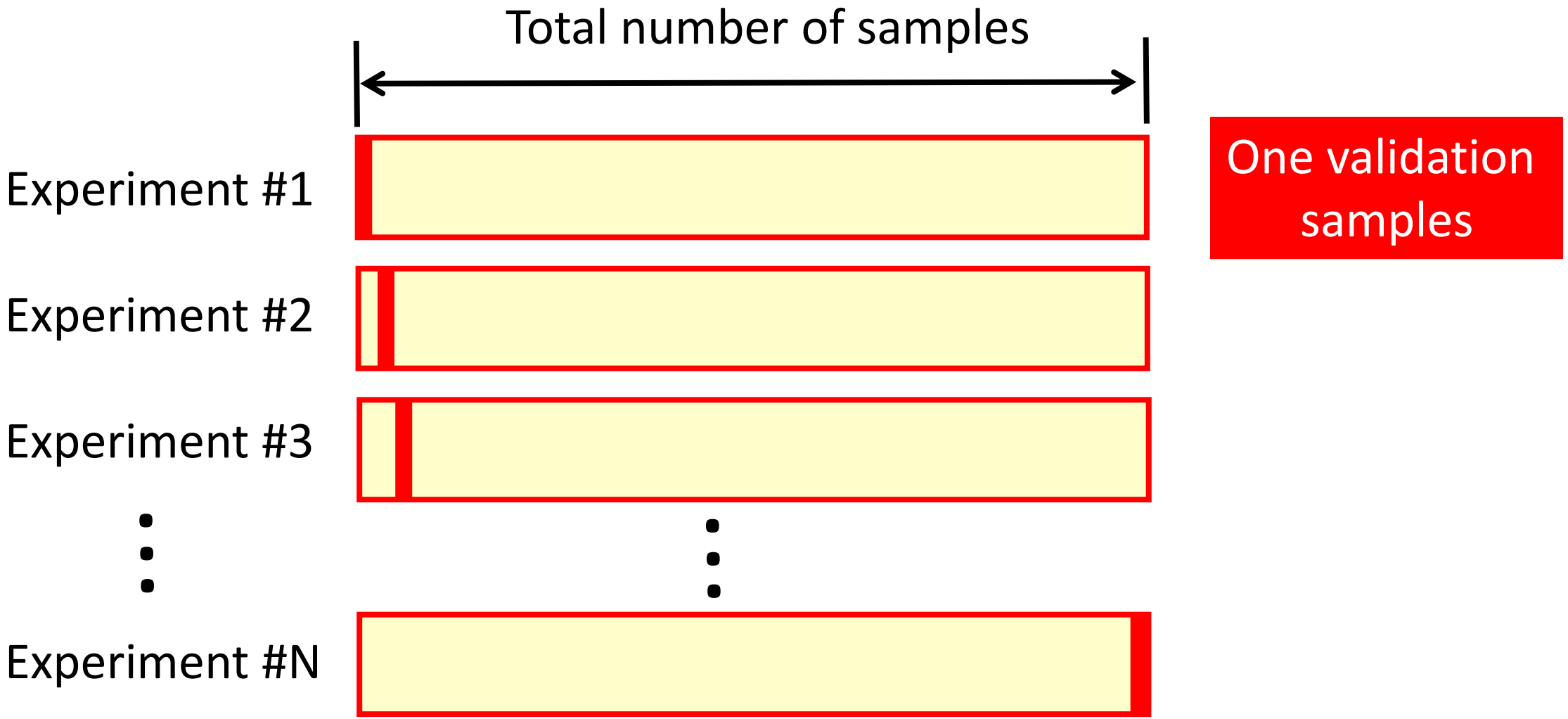
Reference: LECTURE 13: Cross-validation, Introduction to Pattern Analysis, Ricardo Gutierrez-Osuna, Texas A&M University

Random Subsampling



Reference: LECTURE 13: Cross-validation, Introduction to Pattern Analysis, Ricardo Gutierrez-Osuna, Texas A&M University

Leave one out cross validation



Reference: LECTURE 13: Cross-validation, Introduction to Pattern Analysis, Ricardo Gutierrez-Osuna, Texas A&M University

Iris example

iris_cross_validation (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help

Run Code

Iris cross validation

In [1]:
```
import numpy as np
import pandas as pd
import keras
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import np_utils
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import KFold
```

Iris example

```
from sklearn.model_selection import KFold
```

sklearn.model_selection.KFold

class sklearn.model_selection. **KFold** (*n_splits='warn', shuffle=False, random_state=None*) [source]

K-Folds cross-validator

Provides train/test indices to split data in train/test sets. Split dataset into k consecutive folds (without shuffling by default).

Each fold is then used once as a validation while the k - 1 remaining folds form the training set.

Iris example

Parameters:

n_splits : *int, default=3*

Number of folds. Must be at least 2.

Changed in version 0.20: `n_splits` default value will change from 3 to 5 in v0.22.

shuffle : *boolean, optional*

Whether to shuffle the data before splitting into batches.

random_state : *int, RandomState instance or None, optional, default=None*

If int, random_state is the seed used by the random number generator; If RandomState instance, random_state is the random number generator; If None, the random number generator is the RandomState instance used by np.random. Used when `shuffle` == True.

Iris example

In [2]:
```
iris = pd.read_csv("Iris.csv")
dataset = iris.values
X = dataset[:,1:5].astype(float)
Y = dataset[:,5]
iris.shape
```

Out[2]:
```
(150, 6)
```

In [3]:
```
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
inv_encoded_Y = encoder.inverse_transform(encoded_Y)
answer_Y = np_utils.to_categorical(encoded_Y)
```

Iris example

```
def baseline_model():
    model = Sequential()
    model.add(Dense(9, input_dim=4, activation='relu'))
    model.add(Dense(9, activation='relu'))
    model.add(Dense(3, activation='softmax'))
    model.compile(loss='categorical_crossentropy', \
                  optimizer='adam', metrics=['accuracy'])
    return model
```

Iris example

In [5]:
```
model = baseline_model()
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_1 (Dense)              (None, 9)                 45
_________________________________________________________________
dense_2 (Dense)              (None, 9)                 90
_________________________________________________________________
dense_3 (Dense)              (None, 3)                 30
=================================================================
Total params: 165
Trainable params: 165
Non-trainable params: 0
_________________________________________________________________
```

Iris example

```
Fold_number = 10
```

```
kf = KFold(n_splits=Fold_number,random_state=12, shuffle=True)
kf.get_n_splits(X)
print(kf)
```

```
KFold(n_splits=10, random_state=12, shuffle=True)
```

Iris example

```
In [8]: for train_index, test_index in kf.split(X):
            print("TRAIN:", train_index, "\n\nTEST:", test_index, '\n\n')
```

```
TRAIN: [  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  17   1
8
  19  20  22  24  25  26  27  28  29  30  32  33  34  35  36  37  41  42
  43  44  45  46  47  48  49  51  52  53  54  55  56  57  58  59  60  61
  62  63  64  65  67  68  69  70  71  72  73  74  75  76  77  78  79  80
  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96  97  98
 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 117 118
 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 136 137
 138 139 140 141 142 144 145 147 149]

TEST: [ 16  21  23  31  38  39  40  50  66  99 116 135 143 146 148]
```

Iris example

```
scores = []
i = 0
for train_index, test_index in kf.split(X):
    i+=1

    X_train, X_test = X[train_index], X[test_index]
    Y_train, Y_test = answer_Y[train_index], answer_Y[test_index]
    model = baseline_model()
    model.fit(X_train,Y_train,epochs=100,batch_size=5,verbose=0)
```

Iris example

```
score_train = model.evaluate(X_train,Y_train,verbose=0)
score_test = model.evaluate(X_test,Y_test,verbose=0)
```

Iris example

```
    print("Processing fold {:d},".format(i),"train:{:0.2f},".format\
          (score_train[1]),"test:{:0.2f}".format(score_test[1]))
    scores.append(score_test[1])
```

```
Processing fold 1, train:0.98, test:1.00
Processing fold 2, train:0.98, test:0.93
Processing fold 3, train:0.97, test:1.00
Processing fold 4, train:0.98, test:1.00
Processing fold 5, train:0.99, test:0.93
Processing fold 6, train:0.99, test:0.93
Processing fold 7, train:0.99, test:0.93
Processing fold 8, train:0.98, test:0.93
Processing fold 9, train:0.97, test:1.00
Processing fold 10, train:0.98, test:1.00
```

Iris example

```
In [10]: print("Cross Validation with {:d}".format(Fold_number),\
               "folds.\nCV score = {:0.3f} +/- {:0.3f}".\
               format(np.mean(np.array(scores)),\
                      np.std(np.array(scores))))
```

```
Cross Validation with 10 folds.
CV score = 0.967 +/- 0.033
```

Thank you for your attention